Botsync GPM Case: Analysis and Six-Month Roadmap

Oct 31, 2026 · @Bishwarup Das

What is in the submission

- **botsync_data_template.xlsx** is the data template: one tab per table (30 tables). Each column says why it is there, which decision it serves and which table it joins to.
- **botsync_dummy_data.xlsx** holds dummy data for about 250 robots at 30 sites and 18 customers, with 30 days of incidents and charging. The patterns in it are seeded on purpose, as the case asks.
- **Review tabs in that file.** `Analysis` holds every number in this document. Its section 13 tests what the past can predict. Each one is a formula over the data tabs, so you can click a number and see how it was worked out. I cite it as "Analysis 4" for section 4 of that tab. `Assumptions` lists what I assumed, `Planted_patterns` lists the patterns I seeded, `Validation` checks that the tables agree with each other, and `README` explains how the file was built.
- **Data tabs, by what they hold.** Who and where: `customers`, `sites`, `robots`, `firmware_versions`, `stations`, `error_codes`. Battery and time: `battery_packs`, `charge_sessions`, `robot_daily_utilization`. Failures: `incidents`, `interventions`, `recovery_attempts`, `support_tickets`, `fleet_events`, `missions`, `component_telemetry`, `component_replacements`. Site conditions: `site_environment_log`, `site_deployments`, `integration_events`, `network_events`, `traffic_events`, `fleet_density_snapshots`. Cost: `downtime_cost`, `customer_requests`. AI workflow: `incident_evidence`, `change_requests`, `eval_results`, `ai_use_cases`, `ai_flow_steps`.

1. Business goal

Botsync can make one significant MAG1400 Fork improvement in six months. The goal is to raise delivered pallet moves per robot across about 250 robots at 30 sites, so customers renew and expand. The installed base is growing and every new site brings new failure modes, so the improvement should go where it returns the most robot time, not where the complaints are loudest.

2. Who feels it

Who	Needs from the robot	How a failure hurts	Data tabs
Operations head (buyer)	Pallets on schedule, predictable uptime	Missed dispatch, SLA penalty	customers, downtime_cost
Floor supervisor and operators	Few interruptions, easy recovery	Shift spent unblocking robots	incidents, interventions, fleet_events
Site IT	Clean links, no network surprises	Handshake failures, blamed for dead zones	network_events, integration_events
Field technician and support (Botsync)	Clear fault cause, parts known in advance	Travel for no fault found	support_tickets, component_replacements
Deployment engineer (Botsync)	Fast, repeatable go-live	Mapping rework	site_deployments
Product and engineering (Botsync)	A ranked, trustworthy picture of where the robot falls short	Six months spent on the wrong problem	all tables, change_requests

A fix that helps the operations head but adds work for the floor supervisor will be rejected on the floor.

3. Their pain points

What each group runs into, and where it shows in the data. Numbers come from the Analysis tab.

#	Pain point	Evidence in the data	Who feels it
1	Robots spend too long charging	15% of 134,304 scheduled hours, against 1% in failure. 20.4% at the 24-hour sites against 11.6% at the 16-hour sites (Analysis 1, 4)	Operations head, supervisor

#	Pain point	Evidence in the data	Who feels it
2	Docking and pallet-pocket misses	29% of incidents but 14% of downtime hours (Analysis 1, 2)	Supervisor, operator
3	Repairs that need a technician	8% of incidents but 59% of downtime hours (Analysis 1)	Operations head, field technician
4	Dirty lidar and lost localisation	Lidar dirt is 129 incidents and 217 hours. All perception faults together are 644 hours (Analysis 2)	Supervisor, technician
5	Operator and remote effort	Of 801 hours on faults a recovery action could handle, 420 were operator or remote work (Analysis 9)	Supervisor, operator
6	Slow and costly support	Technicians take 168 to 332 minutes to arrive. 158 of 1,033 engineering hours go on no-fault-found tickets, 19% of all tickets (Analysis 6, 7)	Field technician, product and engineering
7	Customers at risk	4 customers flagged churn risk hold ₹1,074 lakh, 25% of contract value. In 30 days customers bear ₹41 lakh and Botsync ₹54.5 lakh (Analysis 1, 8)	Operations head, Botsync

Noise, not pain. Wi-Fi dropouts are frequent (1,784) but only 1 stopped a robot, and traffic waiting totals 3.7 hours (Analysis 7).

4. What the data says

Robots spend 62% of scheduled time moving, 15% charging and 1% in failure (Analysis 1).

1. **Charging, not failure, is the biggest loss.** In 30 days robots spent 20,295 hours charging and 1,319 hours in failure. It concentrates at the 10 sites that run 24 hours: 20.4% of scheduled time charging against 11.6% at the 16-hour sites, and 2.99 emergency low-battery stops per robot against about 0.26 (Analysis 4). The excess is 4,655 hours a month, about 63 per robot.
2. **Ageing packs make it worse.** 57 packs are under 85% health. Their robots stop for low battery 2.70 times each, against 0.59 on healthier packs (Analysis 5).

3. **The loudest failure is not the costliest.** Docking timeouts are 29% of incidents but 14% of downtime hours. Technician-handled repairs are 8% of incidents and 59% of hours. Dirty lidar is 6% of incidents but 13% of hours (Analysis 1, 2).
4. **Firmware or site?** Docking timeouts per 100 robot-days average 11.3 at glare-door sites against 7.3 elsewhere. The glare gap is about 1.0x on the oldest release and 3.5x on the newest (Analysis 3).
5. **Customers.** Ironbridge Castings (12.0 downtime hours per robot) and Harbor Cold Chain (9.1) lose the most time per robot. Four customers are flagged churn risk and hold ₹1,074 lakh, 25% of contract value (Analysis 8). In 30 days customers bear ₹41 lakh in SLA penalties and idle labour, Botsync spends ₹54.5 lakh on support, and 3,868 pallet moves are lost. The rupee rates are assumed (Analysis 1).
6. **Failures cost time more than deliveries.** 98.1% of missions complete, but 41% of incidents in the 7-day extract hit a mission in progress (Analysis 7).
7. **Some things are noise.** Of 1,784 network dropouts only 1 stopped a robot, and traffic waiting totals 3.7 hours in the 7-day extract (Analysis 7).

5. What I would not trust without checking

1. **Why robots fail.** More than half the incidents have no confirmed root cause, so I am reading cause off the error code and the site. Before building on that, I would ask an engineer to confirm cause on a sample and see whether the confirmed incidents look like the rest.
2. **Firmware.** Newer releases look worse at docking, but they are also the ones running at glare-door sites. I would not blame firmware until I had compared releases at the same site.
3. **Downtime hours.** Incident hours and the utilisation tab do not add up (1,729 against 1,319). I would find out why before quoting any hours to a customer.
4. **Customer availability and cost.** SLA uptime is not defined anywhere in the data, and the rupee rates are my assumptions. The ranking of causes should hold. The size of the cost should not be quoted.
5. **It is all dummy data.** The patterns were planted. What this shows is how I would read the data, and I would run the same cuts on real data to see which findings hold. The forecasts in section 6 hold partly because the data gave each site and robot a steady rate.

Also checking: small groups (some sites have fewer than 8 robots; Australia has only 23 technician visits), the charger inventory (there is no chargers tab, so charger counts come from the sessions), and the 7-day event tabs against the 30-day incident tab.

What I would need and do not have (Analysis 12): a catalogue of recovery actions with a record of which one worked on each incident; a code-level cause per incident; swap time, spare-pack needs and handling damage for batteries; a charger inventory per site; an SLA definition and a real downtime cost; and telemetry and video from before an incident, with a longer telemetry history than 7 days to test part-wear forecasts.

6. Options, and how each would work

The pain points point to five directions. I weighed each against the hours it could return.

- **Charging time.** The biggest pool: 4,655 hours a month above the 16-hour sites' rate, at the 24-hour sites (Analysis 4). The option is battery swapping.
- **Faults a recovery action could handle.** 1,475 incidents and 801 hours at level 1, 323 incidents and 258 hours at level 2 (Analysis 9). The option is an LLM recovery and tuning agent.
- **Unclear causes and slow support.** 54% of incidents have no confirmed cause, and 158 engineering hours go on no-fault-found tickets (Analysis 1, 7). The options are LLM triage and a vision model that reads the incident picture.
- **Hardware wear and safety stops.** 556 and 113 hours (Analysis 9). Technician and site work, not a software feature.
- **Wi-Fi and traffic.** Noise (Analysis 7).

The LLM and the vision model come in only for the middle two. Each option is described below.

The three levels of fix

The research gives a ladder from narrow and fast to wide and slow. It never names the levels; the grouping is my reading.

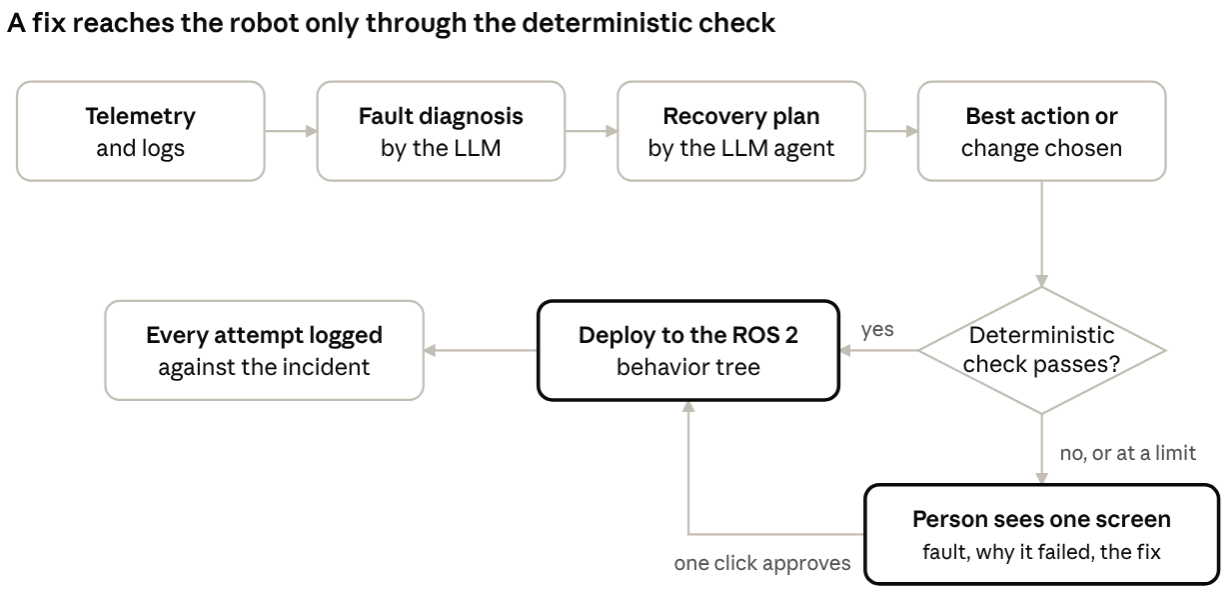
	1. Pick a recovery action (at runtime)	2. Update a Behavior Tree or parameter (offline)	3. Change node code (offline, in simulation)
What the LLM may change	Choose and sequence actions from a whitelist, no new actions. Docking timeout: clear the costmap, back up 1.5 m, rotate 15 degrees, retry.	Edit the tree or a tuned parameter for one site or the fleet. Example: change the recovery order or exposure at a glare site.	Draft a change to a C++ or Python ROS 2 node.

	1. Pick a recovery action (at runtime)	2. Update a Behavior Tree or parameter (offline)	3. Change node code (offline, in simulation)
Gate before it runs	Deterministic evaluator: action whitelist, speed under 0.3 m/s, collision check. A fail falls back to the standard recovery.	Gate 1 on the edit (typed edit, allowlist, value ranges), Gate 2 on the result (replay of past incidents, regression, boundary diff), then a canary site.	The same two gates plus simulation. Never on the vehicle on the fly.
Human role	None for safety. Outcomes reviewed afterwards.	Reviews quality.	Must approve.

The safety layer (safety LiDAR fields, e-stop, speed limits) is never edited by the LLM.

How much each could address (Analysis 9): Level 1 codes are 1,475 incidents and 801 hours, of which 420 hours were operator or remote work. Level 2 codes are 323 incidents and 258 hours. Hardware (168 incidents, 556 hours) and safety stops (171 incidents, 113 hours) sit outside every software level. Level 3 cannot be sized, because no incident has a code-level cause. I assigned the fix levels by hand.

When a gate fails, or a value sits at its limit, nothing deploys. A person gets one screen: the fault, why the change was rejected, and the change that would pass. One click applies it. Inside the limits, no person is needed.



fix flow, from logs to the behavior tree, with the person's route

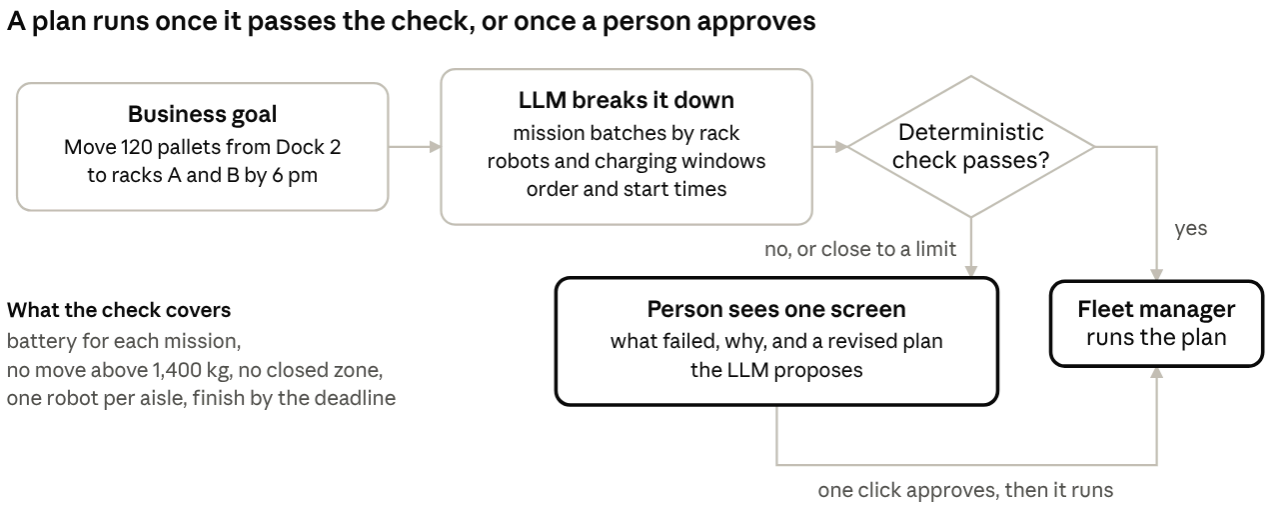
Three jobs for the LLM agent

The levels say how big a change the LLM may make. A second list says what job it does. Both apply together: each job uses the gates above for anything it changes. Build order is triage first, then fleet instructions, then task planning.

1. **Root-cause triage (first).** The LLM reads error logs and unstructured telemetry across the 250 robots and summarises why incidents happened, for the maintenance team. It only reads and suggests, so it carries no risk. It also records confirmed causes, which the fix flow needs: 54% of incidents have none today.
2. **Fleet instructions (second).** Short runtime changes, such as "reroute bot 5 to bay 4 and move everything to bay 6". The deterministic check confirms the bay exists and is open, and that no other robot's route clashes.
3. **Task planning (third, automated).** The LLM turns a business goal into a sequence of robot missions. Worked example below.

Task planning: one example, end to end

Goal (illustrative numbers): move 120 inbound pallets from Dock 2 to racks A and B before the 6 pm truck.



After the run, the plan and its outcome are logged, and planned against actual is stored.

task planning, from business goal to execution

1. **Break it down.** The LLM splits the goal into mission batches (60 pallets to rack A, 60 to rack B, heaviest first), assigns robots and charging windows (R12 and R14 are charging, R07 is low, so it gets a swap before its run), and orders the missions with start times.
2. **Deterministic check.** Hard limits only: enough robots at enough battery for each mission, no move above 1,400 kg, no mission through a zone that is closed or has people working, no two robots in the same aisle at once, and the plan finishes before the deadline.
3. **Pass.** The plan goes to the fleet manager and runs. The plan and its outcome are logged.
4. **Fail, or close to a limit** (a plan that needs a robot under the battery threshold, or finishes with five minutes to spare). A person sees one screen: what failed, why, and a revised plan the LLM proposes. They approve with one click, or edit it. Then it runs.
5. **After the run.** Planned against actual is stored, so the limits can be tuned.

No data in the dummy set covers this job (there are no orders or warehouse-system records), so it stays the last and least certain of the three.

Battery swapping (no LLM)

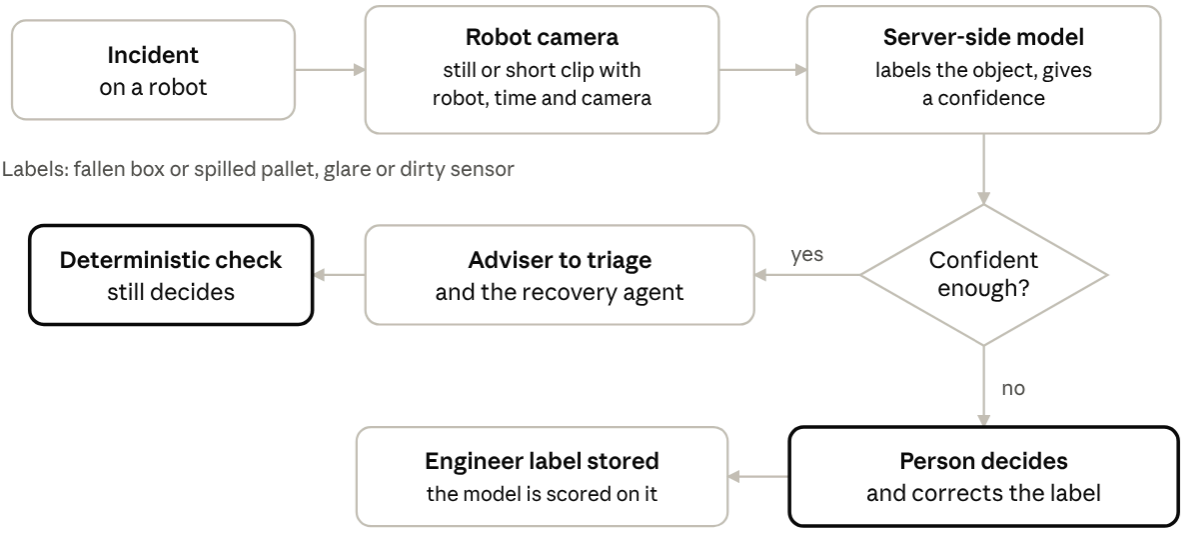
- **Phase 1, person in the loop.** The robot drives to a swap bay at a planned low-charge trigger, not at the emergency stop. An operator swaps the pack with a lift aid, and the spent pack goes to a charging rack.
- **Phase 2, automated.** A swap station exchanges the pack mechanically when the robot docks. I would choose this over a second robot carrying packs, which is much harder to build.

- **Why.** If every charge became a 5-minute swap, up to about 19,300 robot-hours a month could return across the fleet, 14% of scheduled time (Analysis 10). That is a ceiling, not a forecast.
- **Gate to phase 2.** Real swap time, spare packs needed per robot, and pack handling damage.

Vision (VLM)

Two places for it. The first is the robot's own camera: an image is captured at the incident and a server-side model names the object, which needs no new hardware. The second is fixed cameras around the site that watch the whole floor, which cost money at every site, so they come later and only at the worst sites. Naming the object sets the action: a fallen cardboard box can be avoided or moved aside, while a spilled pallet means stop and call a person. The model only advises, and the deterministic check still decides. Its labels match human labels 81% of the time in the dummy evidence set (Analysis 7), and it confuses glare with a dirty sensor, so it starts as an adviser.

The vision model advises, and a person decides when it is unsure



It starts as an adviser. Onboard models and fixed cameras around the site come later.

vision model path, from incident to stored label

Predicting what will fail (no LLM)

Past data says a good deal about what is likely to happen, and it does so with plain statistics, cheaper and safer than a model. I tested the simplest forecast, "the same as before": learn from the first 20 days, check against the last 10 (Analysis 13). Any better method has to beat it.

- **Charging load.** By shift pattern, the charging share in the last 10 days is within 0.2 points of the first 20, and the biggest gap at any single site is 0.6 points (Analysis 13a). That is enough to plan swap-bay staff and spare packs by site, the way past demand decides how many nurses a shift needs.
- **Which robots run flat again.** 48% of robots that had an emergency low-battery stop in the first 20 days had another in the last 10, against 5% of robots with none (Analysis 13b). That is a short list for pack checks and the first swap slots.
- **When docking fails.** At glare-door sites 51% of docking timeouts start between 14:00 and 17:00, against 21% at other sites (Analysis 13c). The exposure setting can be applied in that window, or robots kept off those bays.
- **Which robots fail again.** About a third of incidents (34%) repeat within 7 days. The 37 robots with 12 or more incidents in the first 20 days, 15% of the fleet, produced 42% of the incidents after (Analysis 13d).
- **Component wear.** This is the best fit for the 168 to 332 minute technician delay, because a part flagged early can be fixed on a planned visit. But telemetry covers only 7 days (Analysis 13e), so how far ahead a failure can be flagged cannot be tested here. It needs a longer history.

How I would calculate it. Start with the same-as-before forecast. Then count incidents by site, hour and error code over past weeks. Then, if that is not enough, a robot-level model of who fails next. Each is trained on earlier days and judged on later days it has not seen, by how many real failures sit in the top 10% of robots flagged and how many false alarms it costs. A method is accepted only if it beats same-as-before.

What this cannot do. The data was built with a steady rate for each site and robot, so these results hold partly by design, and real fleets drift. It is one 30-day window with no weekly or seasonal pattern. Nothing here predicts an unusual event, and a new firmware release, site or layout breaks a forecast until it is re-tested. With no order data, there is no demand forecast. The LLM does not do any of this; it only reads the results.

7. What each capability needs

Battery swapping

- A pack one operator can swap in about 10 minutes (target, assumed): handles, a latch, a quick connector and a lift aid. Pack weight is not in the data, so engineering has to confirm it.
- A swap at a planned low-charge level, at a marked bay near the work area, before the robot reaches an emergency stop.

- Spare packs. With an assumed 8-hour run and a 100-minute charge, about 0.3 spare packs per robot: roughly 22 for the 74 robots at 24x7 sites, and about 6 for a 19-robot pilot.
- A rack with enough chargers for the spare packs.
- A swap record: robot, pack out, pack in, start, end, operator. It sits next to the pack tab, and pack health decides which packs are retired.
- Done when swap time, emergency low-battery stops and charging share are measured against 24x7 sites that have not switched.

LLM

- **Triage.** Reads incidents, recovery attempts, telemetry and tickets, and returns a likely cause with its evidence and a confidence. An engineer confirms or corrects it in one click, and the confirmed cause is stored.
- **Recovery agent.** Chooses from a whitelist of recovery actions, passes the deterministic evaluator, and falls back to the standard recovery when it fails. A failed or at-the-limit change goes to the one-click screen for a person.
- **Fleet instructions and task planning.** As described above.
- **Data it needs and does not have.** A catalogue of recovery actions, a record of which action worked on each incident, confirmed causes (only 46% of incidents have one today), and fix-effort baselines.
- The safety layer is never editable by the LLM.

VLM

- A still or short clip at every incident, with robot, time and camera, sent to a server-side model.
- A label (fallen box or spilled pallet, glare or dirty sensor) with a confidence. Below a set confidence, a person decides.
- It starts as an adviser and is scored against engineer labels. Onboard models and fixed cameras around the site come later.

Forecasting

- A weekly job on the charge sessions, incidents and daily utilisation tabs. The first four forecasts in section 6 need no new data.
- Outputs: charge starts per hour by site (sets the swap-bay rota and the spare packs), the robot watch list, and the docking window at each glare-door site.
- Each forecast is scored against "the same as before" on later days, reviewed monthly, and re-tested after any new firmware release, new site or layout change.

- Needs that the data does not meet: a telemetry history longer than 7 days for part wear, and order data for any demand forecast.

8. Prioritisation (RICE)

Item	Reach (robots)	Impact	Confidence	Effort (person-months)	RICE score	Hours at stake per month
LLM root-cause triage	250	0.25	80%	4	12.5	158 engineering hours on no-fault-found tickets (Analysis 7)
LLM fleet instructions (reroute)	250	0.25	50%	4	7.8	about 90 (blocked path and deadlock, Analysis 2)
Battery swap, phase 1 (24x7 sites)	**74**	**2**	**50%**	**10**	**7.4**	**4,655 above the 16-hour sites' charging rate (Analysis 4)**
VLM evidence and labels (advisory)	250	0.25	50%	5	6.3	644 in perception faults it could help diagnose (Analysis 2)
LLM Behavior Tree or parameter changes	250	0.25	50%	8	3.9	258 (Analysis 9)
LLM recovery agent (level 1)	250	0.25	50%	9	3.5	801, of which 420 operator or remote work (Analysis 9)
Battery swap, phase 2 (automated station)	74	3	30%	30	2.2	up to 10,300 for 24x7 sites if every charge became a swap (Analysis 4)
LLM task planning	250	0.25	20%	10	1.3	no data
Fixed cameras around sites	100	0.25	30%	12	0.6	not sized

Impact is the share of scheduled time returned to the robots reached: 3 is over 10%, 2 is 5 to 10%, 1 is 2 to 5%, 0.5 is 0.5 to 2%, 0.25 is under 0.5%. The 4,655 hours is 8.7% of the 24x7 sites' scheduled time. Effort and confidence are my estimates, so treat the scores as a check on the argument, not as proof.

RICE puts swap third, behind triage and fleet instructions. Those two win because they are cheap. Neither returns robot time directly: the hours at stake are 158 engineering hours and about 90 robot-hours. I am using RICE as a check and choosing on the size of

the prize.

9. The recommendation

The one improvement for the next six months: battery swapping, phase 1, with a person doing the swap, at the 10 sites that run 24 hours.

How the data led there:

1. Charging takes 15% of scheduled time and failures take 1%. Total incident downtime across the fleet is 1,729 hours in 30 days. Charging time is 20,295 (Analysis 1).
2. The loss is concentrated. The 24x7 sites (10 sites, 74 robots) spend 20.4% of scheduled time charging against 11.6% at the 16-hour sites. The 4,655 hours above the 16-hour rate is more than seven times the 644 hours lost to all docking, lidar, pocket and localisation faults put together. Those sites also average 2.99 emergency low-battery stops per robot, against about 0.26 (Analysis 2, 4).
3. Packs make it worse. 57 packs are under 85% health, and their robots stop for low battery 2.70 times each, against 0.59 (Analysis 5).
4. The ceiling is about 19,300 robot-hours a month across the fleet, or 14% of scheduled time, if every charge became a 5-minute swap (Analysis 10). That is a ceiling, not a forecast. My phase 1 target is smaller: take back about half of the 4,655 hours, roughly 2,300, or 1.7% of fleet scheduled time.

What argues against it: swapping only helps the 24x7 sites, 10 of 30 sites and 74 of 250 robots. The other sites already charge in their rest hours, at 11.6% of scheduled time. Swapping also needs a pack redesign and a new process on the floor, which software does not.

What follows it, in order:

1. **Triage** starts in month 1 on a small team, because it is cheap and creates the confirmed causes the rest of the LLM work needs.
2. **VLM evidence and labels**, advisory only.
3. **Recovery agent (level 1)**, first in shadow mode, then at two canary sites behind the deterministic evaluator.
4. **Automated swap station (phase 2)**, only if the month-4 gate passes.

What I would deliberately not do in this window:

- **Task planning.** There is no order or warehouse data to base it on.
- **Automated swap station before phase 1 has run.** It costs about three times as much, and swap time and spare packs are still guesses.

- **Fixed cameras around sites.** Costly at every site, and the onboard camera answers the same questions first.
- **Code changes by the LLM (level 3).** There is no code-level cause for any incident.
- **Hardware wear.** It is 168 incidents but 556 hours, and it is technician work for the service team, not a software feature.
- **Safety stops.** 171 e-stops and 113 hours, mostly a matter of site traffic.
- **Wi-Fi and traffic gridlock.** 1 of 1,784 network dropouts stopped a robot, and traffic waits in the 7-day extract total under 4 hours (Analysis 7).

What would change my mind: a real swap taking much longer than 10 minutes, a spare-pack need well above 0.3 per robot, or a pack that cannot be handled safely by one person. Any of these would move the first investment to triage and the recovery agent.

10. Which models to use

Customer sites, their incident pictures and fault logs should not go to an outside provider, so no third-party hosted LLM or VLM. Both jobs can run on small open models that Botsync hosts and tunes itself. The model names below are examples to benchmark, not final picks.

Model	Option	Why	Risk and how I would handle it	My pick
LLM	Third-party hosted model	Strongest out of the box	Fault data and site details leave Botsync's control	No
LLM	Small open model (7 to 8 billion parameters), fine-tuned on confirmed causes and recovery outcomes, hosted by Botsync	Cheap to run, data stays in-house, enough for triage and picking from a recovery whitelist	Needs labelled data, and only 46% of incidents have a confirmed cause today. It can still be wrong, so every output goes through the gates in section 6, and a fixed set of past incidents is replayed after each retrain	Yes

Model	Option	Why	Risk and how I would handle it	My pick
LLM	Large open model, self-hosted, offline only	Better for drafting behavior tree and code changes in simulation	GPU cost, slow, and those jobs are the least certain	Later, if needed
VLM	Third-party hosted model	Best out of the box	Pictures of customer premises, often with people in them, leave Botsync's control	No
VLM	Small open model (2 to 7 billion parameters), fine-tuned on engineer-labelled incident pictures, hosted by Botsync	Runs on one server, no new hardware, advisory first	Dummy labels match 81% and glare is confused with a dirty sensor. It starts as an adviser scored against engineer labels. Faces and badges are blurred before storage	Yes
VLM	Tiny model on the robot	Works with no network, and fastest	Limited compute and heat on the robot, and it cannot be tuned well without the server model's labels first	After the server model works

How I would choose between the small models: run two or three on the same 200 or so engineer-labelled incidents, and compare accuracy, response time and memory against the 81% match in the dummy set. The cheapest one that clears the bar wins.

11. Six-month roadmap

Month	Battery swap (the one improvement)	LLM and VLM (follows it)	Gate or decision
1 (Nov)	Pick three pilot sites: S08, S21, S22 (19 robots, `sites` tab). Define pack, bay and swap steps. Confirm pack weight and the lift aid. Start recording swaps.	Triage on existing logs; engineers confirm the cause on each case. Standard picture capture at every incident.	Baseline charging share and emergency low-battery stops at pilot and unchanged 24x7 sites.
2	Build bays and a rack, buy about 6 spare packs, train operators. Swaps go live at S08.	Triage live for the maintenance team. Draft a list of recovery actions from past recovery attempts.	Swap time measured on the first 50 swaps.
3	Swaps live at all three pilot sites.	The model labels objects at incidents (advisory), scored against engineer labels.	
4	Extend to the other seven 24x7 sites if the gate is passed.	Recovery agent in shadow mode: it proposes, operators act, and the two are compared.	Gate: swap time near the 10-minute target, charging share at pilot sites well below unchanged 24x7 sites, no pack damage.
5	All 24x7 sites (74 robots, about 22 spare packs).	Recovery agent live at two canary sites behind the evaluator and fallback.	
6	Measure against the targets below.	Fleet instructions in shadow mode.	Decide: automated swap station, 16-hour sites, and how far the LLM work goes.

Targets for the swap (assumed, to be set in month 1): charging share at pilot sites from about 23% to under 14%, emergency low-battery stops per robot from 2.99 to under 1, no pack damage.

Forecasting runs alongside the swap, with no new hardware. Month 1: the same-as-before baseline from Analysis 13 for charging load by site and hour, which sets the swap-bay rota and the number of spare packs. Month 2: the weekly watch list of robots and the 14:00 to 17:00 docking window at glare sites. Month 4: compare a model against the baseline on later days, and keep it only if it wins.

How this helps customers

- **More working hours where it hurts.** If the 4,655 excess hours went away, the 24x7 sites would get about 63 more working hours per robot per month.
- **Fewer surprise stops.** Emergency low-battery stops drop at the sites that have the most of them.
- **Faster, better fixes.** Triage points maintenance at the cause, so there are fewer wasted visits (19% of tickets end as no fault found). Technicians take 168 to 332 minutes to arrive depending on region, so a correct remote answer saves the most time where they are slowest.

How this helps Botsync

- **More deliveries per robot without selling more robots.**
- **Less support cost.** Support spend is about ₹54 lakh in 30 days at assumed rates, and 158 engineering hours go on tickets that find no fault.
- **Renewals.** Four customers are flagged churn risk and hold 25% of contract value (₹1,074 lakh). Time returned to the busy sites is the most direct thing to offer them.
- **Data advantage.** The swap record and the confirmed causes are data Botsync does not have today, and the LLM work needs both.